
Project III

Oxford

Pre-Requisites:

- 18. Function Aliasing
- 19. enumerate() built-in function
- 20. Dictionaries & Dictionary Methods
- 21. String Formatting
- 22. Asterisk (*) operator
- 23. *args and **kwargs
- 24. map() and filter() built-in functions & “lambda” keyword
- 25. Writing and Reading from a file & “with” keyword
- 26. “import”, “from” and “as” keywords & Common Libraries (os, time, math)
- 27. Exception handling & “try”, “except”, “finally” and “raise” keywords

*Note that the usage of the content learned in pre-requisites varies from implementation to implementation and the solution does not necessarily need to use everything stated above. Moreover, knowing all of the pre-requisites does not guarantee success i.e. sometimes you need to do some research and learn something yourself that is NOT mentioned in the pre-requisites.

Project Description:

You will be writing a semi-library which can be used to create personal dictionaries. This library has 9 functions, which can be categorized into two categories; the first group are the ones that require a path to interact with a specific “file”; while the others will be manipulating the current dictionary. The dictionary’s structure is as followed:

The dictionary’s keys are the words that are saved into it. Each key’s value is a tuple. The first element of said tuple is the word’s plural form, if any; the second element is another dictionary itself. This new dictionary’s keys are PartsOfSpeech (PoS), namely ‘noun’, ‘verb’, ‘adjective’ and ‘adverb’. Each of these keys hold a list. Each element of this list, is one definition. Here is a representation:

```
{word1: (PluralForm, {PoS1: [Definition1, Definition2, ...], PoS2: [...], ...}), word2: (...), ...}
```

As for the functions, the first group’s functions all have two parameters. The first parameter is “name” which is a string and shows the name of the file in which the dictionary values are stored. The second parameter is “path” which is also a string, but is an optional parameter whose default value is the current directory.

Below are the details about each function and what they do:

1. createDict: Creates a file called [name].dict in the [path] directory. The file’s content are:
 - a. The time the file was created, in YY-MM-DD hour:min:sec format
 - b. Last time the file was edited, in YY-MM-DD hour:min:sec format
 - c. The dictionary it holds. The structure of which is given above.

If [name].dict already exists in the given directory, it raises FileExistsError saying ‘[name].dict already exists.’

2. loadDict: Loads the dictionary of the [name].dict in the given directory.
If [name].dict does not exist in the directory, it raises FileNotFoundError saying ‘[name].dict does not exist.’
3. saveDict: Overwrites the dictionary values of [name].dict in the given path with the current dictionary’s values. It also updates the last edited time.
If [name].dict does not exist in the given directory, it creates it.
4. removeDict: Deletes the [name].dict file in the given directory.
If [name].dict does not exist in the directory, it raises FileNotFoundError saying ‘[name].dict does not exist.’

5. dictInfo: Returns a dictionary with 2 keys “Created” and “Edited” which hold values for the creation and last edited date of [name].dict in the given path. If [name].dict does not exist in the directory, it raises FileNotFoundError saying ‘[name].dict does not exist.’

This function has an additional optional parameter named ‘showResult’ whose default value is False. If ‘showResult’ is True, the function will print the creation date and last edited date of the file.

The second group of functions are more different from each other:

6. addWord: This function will get at least 3 parameters. These are “word”, “PoS” and “definitions”. All of these parameters are string. It should be noted that the function can be called with more than 3 values, say 10 values. Then the first two values will be “word” and “PoS” and the rest 8 will be the “definitions”. The function also has an additional “plural” parameter whose default value is “Default”; it accepts either a string or None value type. This function does not return anything.

When called, the function adds the given word to the dictionary. If the word already exists, it checks whether for availability of the given PoS. If it does not exist, it adds the PoS and the given definitions. If it does exist, it adds the new definitions to the end of the definitions list that the PoS has. As for the plural value, if the word already exists, the new plural value overrides the former value as long as it is not None. If no value was given for plural value and it still holds “Default”, the plural value will be made automatically. If the word ends in “s”, “sh”, “ch”, “x” or “z” for plural value “es” will be added to the end of it. If not only “s” will be added.

7. removeWord: This function only has one parameter “word” which is a string. This function does not return anything.

When called, the function removes the given word from the dictionary. If the word doesn’t exist, it doesn’t do anything.

8. searchWord: This function has two parameters “word”, which is a string, and “showResult” whose default value is False.

If the given word exists in the dictionary, it returns its values i.e. the values that the [word] key holds. If it doesn’t exist, it returns None.

If showResult is True, it prints the values as will be shown in **Example Run**.

9. listWords: This function only has one parameter “showResult” whose default value is False. This function returns a list containing all the words in the dictionary in alphabetical order.

If showResult is True, it prints the result as shown in **Example Run**.

In addition to all the given functions, there is one more thing that the library does. Whenever a new word is added to the dictionary, if the word already exists in one of the definitions, two underlines will be added to before and after it. For instance, take the definition for the word “complex” in context of psychoanalysis:

“a related group of repressed or partly repressed emotionally significant ideas which cause psychic conflict leading to abnormal mental states or behaviour.”

Now if we add the adjective “Repressed” to our dictionary the definition becomes:

“a related group of repressed or partly repressed emotionally significant ideas which cause psychic conflict leading to abnormal mental state or behaviour.”

It should be noted that this is NOT case sensitive. Meaning even if we had the word Repressed, it would be underlined.

Moreover, if the definition of the word is removed from the dictionary, subsequently the underlines will be removed as well.

Some additional notes:

1. When printing the result, the order for PoS definitions will be:
 - a) Adjective
 - b) Adverb
 - c) Noun
 - d) Verb
2. addWord function is case sensitive. Meaning “pos” and “PoS” will be two different words.
3. In case there are no words in the dictionary listWord function prints an empty string.

Example Run

```
>>> createDict("Oxford", r"C:\Users\Public")
>>> loadDict("Oxford", r"C:\Users\Public")
>>> addWord("Do", "noun", "[Informal British] a party or other social event.",
"[Informal North American] short for hairdo.", "[Informal] excrement ", "[Informal
Archaic] a swindle or hoax.")
>>> addWord("Do", "verb", "perform (an action, the precise nature of which is
often unspecified).", "achieve or complete.", "act or behave in a specified way.",
"be suitable or acceptable.", "[Informal] beat up or kill.", plural=None)
>>> saveDict("Oxford", r"C:\Users\Public")
>>> addWord("Rhythm", "noun", "a strong, regular repeated pattern of
movement or sound.", "the measured flow of words and phrases in verse or prose
as determined by the relation of long and short or stressed and unstressed
syllables.", "a regularly recurring sequence of events or processes.")
>>> addWord("Fish", "verb", "catch or try to catch fish, typically by using a net or
hook and line.", "search by groping or feeling for something concealed.")
>>> addWord("Fish", "noun", "a limbless cold-blooded vertebrate animal with
gills and fins living wholly in water.", "[Informal British] a person who is strange in
a specified way.", plural="Fish")
>>> addWord("Excrement", "noun", "waste matter discharged from the bowels;
faeces.")
>>> saveDict("Oxford", r"C:\Users\Public")
>>> dictInfo("Oxford", r"C:\Users\Public", showResult=True)
```

Created at: 2024-02-21 12:53:46

Edited at: 2024-02-21 12:54:01

```
>>> listWords(True)
```

1. Do
2. Excrement
3. Fish

4. Rhythm

```
>>> searchWord("Do", showResult=True)
```

"Do" [Plural]: "Dos"

noun

1. [Informal British] a party or other social event.
2. [Informal North American] short for hairdo.
3. [Informal] __excrement__
4. [Informal Archaic] a swindle or hoax.

verb

1. perform (an action, the precise nature of which is often unspecified).
2. achieve or complete.
3. act or behave in a specified way.
4. be suitable or acceptable.
5. [Informal] beat up or kill.

```
>>> searchWord("Fish", showResult=True)
```

"Fish" [Plural]: "Fish"

noun

1. a limbless cold-blooded vertebrate animal with gills and fins living wholly in water.
2. [Informal British] a person who is strange in a specified way.

verb

1. catch or try to catch fish, typically by using a net or hook and line.
2. search by groping or feeling for something concealed.

```
>>> excrementInfo = searchWord("Excrement")
```

```
>>> excrementInfo
```

```
('Excrements', {'noun': ['waste matter discharged from the bowels; faeces.']})
```

Assessment & Grading

- You need to save your file as “oxford.py” and upload it until 17/03/2024 Sunday 11:59pm on <https://forms.gle/wLit9SzG62Cpg85PA>
- All of your functions and details (such as alphabetical order of words, underlining words in definitions and etc...) will be tested and your grade will be calculated based on how many of the functions and details are implemented correctly.
- You are guaranteed that your files will not be given paths that need administrative permission.
- You are guaranteed that if a word is going to be underlined, that word will always have space before and after it.
- You can define functions other than the ones mentioned, however in the testing process only the 9 mentioned functions will be called.
- You can define global variables and use them in your solution.
- You may use built-in libraries if needed.
- You are NOT allowed to use classes or in general OOP.